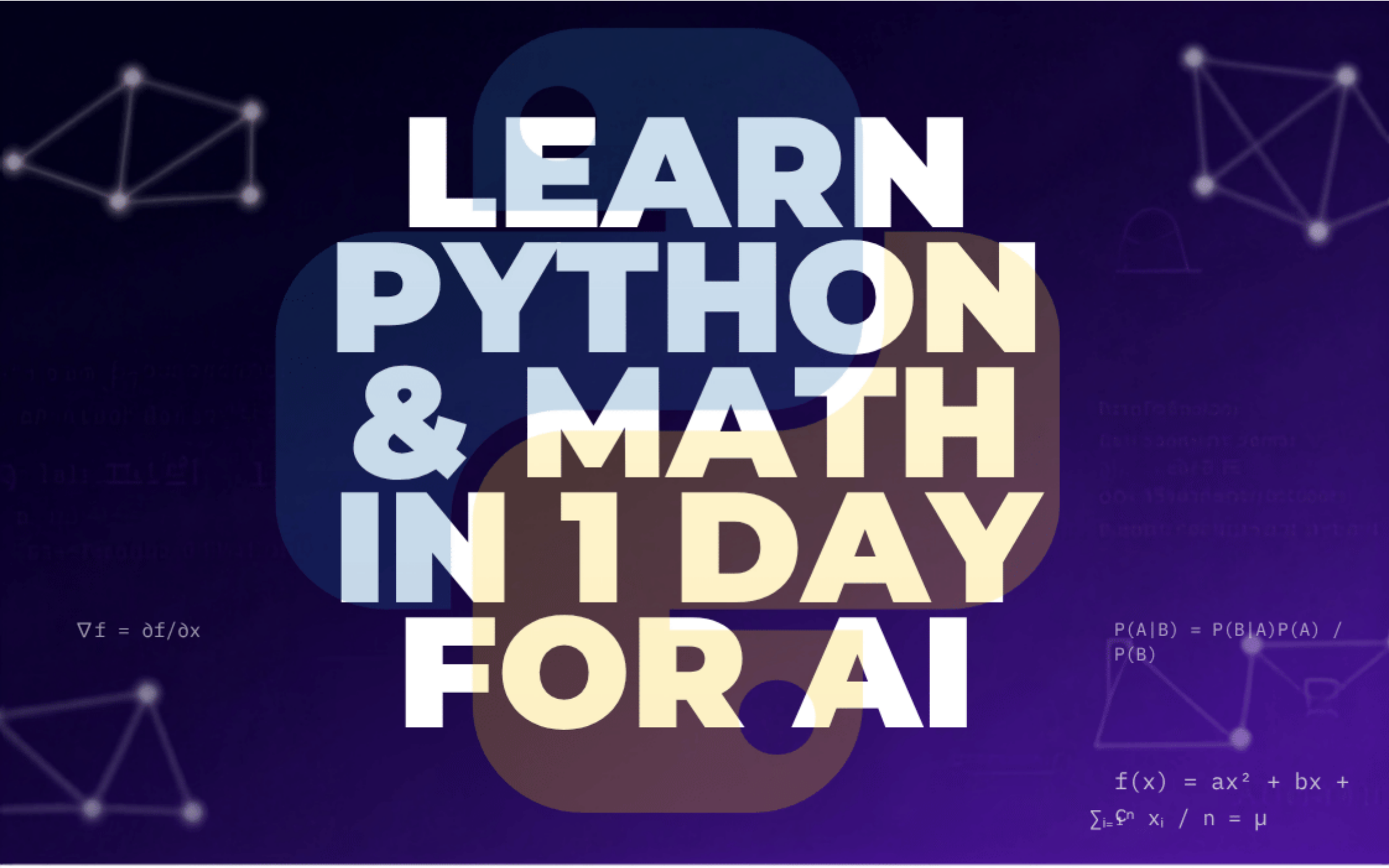


AI-powered roadmap at bimberio.com

Part 2: Math with NumPy

Introduction to NumPy

NumPy (Numerical Python) is the fundamental library for scientific computing in Python. In AI and Data Science, we rarely work with single numbers; instead, we work with large collections of data called **Arrays**.

Why use NumPy instead of standard Python lists?

1. **Speed:** NumPy arrays are written in C, making them significantly faster than Python lists for mathematical operations.
2. **Memory Efficiency:** They consume much less memory.
3. **Vectorization:** You can perform operations (like addition or multiplication) on an entire array at once without writing `for` loops.
4. **Math Tools:** It comes with built-in functions for linear algebra, Fourier transforms, and random number generation.

From Python Lists to NumPy Arrays

What is a NumPy Array?

An **array** is a central data structure of the NumPy library. It is a grid of values and it contains information about the raw data, how to locate an element, and how to interpret an element.

Unlike Python lists, which can hold different types of data (strings, ints, floats) in one list, a NumPy array **must have all elements of the same type**. This 'homogeneity' is exactly what makes NumPy so fast.

In [ ]:

```python
import numpy as np

# Creating a NumPy array from a list
my_array = np.array([10, 20, 30, 40, 50])

print(f"NumPy Array: {my_array}")
print(f"Type: {type(my_array)}")
```


▶ Run in Colab

Python List vs. NumPy Array

While they look similar, they behave very differently:

- **Python Lists:** Designed as general-purpose containers. They are flexible but slow for math because Python must check the type of every single element during an operation.
- **NumPy Arrays:** Designed specifically for mathematical operations. Since every element is the same type (e.g., all 64-bit integers), the computer can process them much more efficiently at the hardware level.

Benefits of NumPy Arrays

Feature	Python Lists	NumPy Arrays
Data Type	Heterogeneous (Multiple types)	Homogeneous (Single type)
Memory	High overhead	Compact/Efficient
Performance	Slower (Looping required)	Fast (Vectorized)
Functionality	Basic methods (append, pop)	Advanced Math (Linear Algebra, Stats)

Vectorization

One of the biggest benefits is **Vectorization**. In a Python list, to add `1` to every number, you must write a `for` loop. In NumPy, you simply write `array + 1`, and NumPy handles the rest instantly.

In [ ]:

```python
import numpy as np

# NumPy Vectorization
arr = np.array([1, 2, 3, 4, 5])

# One single operation applied to all elements
result_arr = arr + 10

print(f"Original Array: {arr}")
print(f"Vectorized Result (arr + 10): {result_arr}")
```


▶ Run in Colab

In [ ]:

```python
# Python List Comparison
my_list = [1, 2, 3, 4, 5]

# To add 10 to every item, we must loop or use a list comprehension
result_list = [x + 10 for x in my_list]

print(f"Original List: {my_list}")
print(f"List Comprehension Result: {result_list}")
```


▶ Run in Colab

1. Scalars

A **Scalar** is a single numerical value. It has a magnitude but no direction or dimensionality. In NumPy, a 0-dimensional array represents a scalar.

- **Example:** `5`, `3.14`, or `np.array(42)`.

In [ ]:

```python
import numpy as np

# Creating a 0-D array (Scalar)
scalar = np.array(42)
print(f"Scalar value: {scalar}")
print(f"Number of dimensions: {scalar.ndim}")
```


▶ Run in Colab

2. Vectors

A **Vector** is an ordered collection of numbers. In physics, it represents a quantity with both magnitude and direction. In NumPy, a 1-dimensional array is considered a vector.

- **Example:** `[1, 2, 3]` is a 3-element vector.

In [ ]:

```
# Creating a 1-D array (Vector)
vector = np.array([1, 2, 3, 4, 5])
print(f"Vector: {vector}")
print(f"Number of dimensions: {vector.ndim}")
```


▶ Run in Colab

3. Dimensions (Rank)

Dimensions (often called **Axes** in NumPy) refer to the levels of nesting in an array:

- **0D (Scalar):** A single point.
- **1D (Vector):** A line of numbers.
- **2D (Matrix):** A table of numbers with rows and columns.
- **3D+ (Tensors):** Cubes or higher-order stacks of data, essential for deep learning (e.g., images with RGB channels).

In [ ]:

```python
# Creating a 2-D array (Matrix)
matrix = np.array([[1, 2, 3], [4, 5, 6]])

# Creating a 3-D array (Tensor)
tensor = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print(f"Matrix (2D) dimensions: {matrix.ndim}")
print(f"Tensor (3D) dimensions: {tensor.ndim}")
print(f"Tensor shape: {tensor.shape}")
```


▶ Run in Colab

4. Tensors

In mathematics and computer science, a **Tensor** is a generalization of scalars, vectors, and matrices to higher dimensions.

- **0D Tensor:** Scalar (a single number).
- **1D Tensor:** Vector (a line of numbers).
- **2D Tensor:** Matrix (a grid of numbers).
- **3D Tensor:** A cube of numbers (like an RGB image with height, width, and 3 color channels).
- **4D Tensor:** Often used in Deep Learning to represent a **batch** of images (batch size, height, width, channels).

In [ ]:

```python
# Creating a 4D Tensor (e.g., batch of 2 images, 3x3 pixels, 1 channel)
batch_images = np.zeros((2, 3, 3, 1))

print(f"Shape: {batch_images.shape}")
print(f"Dims: {batch_images.ndim}")
print(f"Size: {batch_images.size}")
```


▶ Run in Colab

Comparison: 3D vs. 4D Tensors

In Deep Learning, the jump from 3D to 4D is the most common transition you will encounter.

Feature	3D Tensor	4D Tensor
Mental Model	A Cube (Single Object)	A Shelf of Cubes (Batch of Objects)
Example Case	One RGB Image	A Batch of RGB Images
Dimensions	`(Height, Width, Channels)`	`(Batch_Size, Height, Width, Channels)`
Total Elements	$H \times W \times C$	$B \times H \times W \times C$

In [ ]:

```
### 3D Tensor: A single 4x4 pixel RGB image (3 channels)
```


Run in Colab

In [ ]:

```
import numpy as np
single_image = np.random.rand(4, 4, 3)

print("--- 3D Tensor (Single Image) ---")
print(f"Shape: {single_image.shape} (H, W, C)")
print(f"Dims:  {single_image.ndim}")
```


▶ Run in Colab

4D Tensor: A batch of 5 such images

In [ ]:

```
image_batch = np.random.rand(5, 4, 4, 3)

print("--- 4D Tensor (Batch of 5 Images) ---")
print(f"Shape: {image_batch.shape} (B, H, W, C)")
print(f"Dims:  {image_batch.ndim}")
```


▶ Run in Colab

Basic Arithmetic Operations in NumPy

One of the most powerful features of NumPy is its ability to perform arithmetic operations directly on arrays. These operations are performed **element-wise**, meaning the operation is applied to each corresponding pair of elements in the arrays.

1. Array Addition

When you add two arrays of the same shape, NumPy adds the elements at each position together.

In [ ]:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Element-wise addition
result = a + b

print(f"Array A: {a}")
print(f"Array B: {b}")
print(f"A + B:   {result}")
```


▶ Run in Colab

2. Array Subtraction

Subtraction works similarly, subtracting the elements of the second array from the first at each index.

In [ ]:

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])

# Element-wise subtraction
result = a - b

print(f"Array A: {a}")
print(f"Array B: {b}")
print(f"A - B:   {result}")
```


▶ Run in Colab

3. Array Multiplication (Element-wise)

In NumPy, the `*` operator performs element-wise multiplication. Note that this is different from 'Matrix Multiplication' in linear algebra.

In [ ]:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Element-wise multiplication
result = a * b

print(f"Array A: {a}")
print(f"Array B: {b}")
print(f"A * B:   {result}")
```


▶ Run in Colab

4. Array Division

Division divides each element of the first array by the corresponding element in the second array.

In [ ]:

```
a = np.array([10, 20, 30])
b = np.array([2, 4, 5])

# Element-wise division
result = a / b

print(f"Array A: {a}")
print(f"Array B: {b}")
print(f"A / B:   {result}")
```


▶ Run in Colab

5. Matrix Multiplication (Dot Product)

Unlike element-wise multiplication, the **dot product** follows the rules of linear algebra. For two vectors, it results in a single scalar. For matrices, it performs row-by-column multiplication.

- **Method:** `np.dot(a, b)`
- **Operator:** `a @ b` (Introduced in Python 3.5)

In [ ]:

```python
import numpy as np

# 2D Matrices
matrix_a = np.array([[1, 2], [3, 4]])
matrix_b = np.array([[5, 6], [7, 8]])

# Perform Matrix Multiplication
dot_result = np.dot(matrix_a, matrix_b)
# Alternative using the @ operator
matmul_result = matrix_a @ matrix_b

print("Matrix A:\n", matrix_a)
```


▶ Run in Colab

`np.dot()` vs. the `@` Operator

While they appear identical for 2D matrices, there are important technical differences when working with higher-dimensional arrays:

1. `np.dot(a, b)`:
   - Performs a dot product based on the last axis of `a` and the second-to-last axis of `b`.
   - It is more flexible but can produce unexpected shapes when dealing with complex tensors.
2. `a @ b` **(matmul)**:
   - Introduced in Python 3.5 specifically for matrix multiplication.
   - It treats higher-dimensional arrays as **stacks of matrices** (batches) and performs multiplication on the last two dimensions.
   - In Deep Learning, `@` is almost always preferred because it handles batches of data (4D Tensors) correctly by default.

Summary: For basic 2D math, they are interchangeable. For AI and Deep Learning involving batches, use the `@` operator.

Example: `np.dot` with 3D Tensors

When we use `np.dot` on higher-dimensional arrays, the resulting shape is a combination of the non-contracted axes. Notice how the output shape for `np.dot` differs significantly from what you might expect in a batch-processing (Deep Learning) context.

In [ ]:

```python
import numpy as np

# Create two 3D tensors
# Shape: (Batch_Size, Rows, Cols)
A = np.random.rand(2, 3, 4)
B = np.random.rand(2, 4, 5)

# 1. Using np.dot()
dot_res = np.dot(A, B)

# 2. Using @ (matmul)
matmul_res = A @ B
```


Run in Colab

1. Preparation of 3D Tensors

First, we create two 3D tensors with a shape typically found in deep learning: `(batch_size, rows, columns)`.

In [ ]:

```python
import numpy as np

# Shape: (Batch_Size, Rows, Cols)
A = np.random.rand(2, 3, 4)
B = np.random.rand(2, 4, 5)

print(f"Shape of A: {A.shape}")
print(f"Shape of B: {B.shape}")
```


▶ Run in Colab

2. Using `np.dot()` on 3D Tensors

When using `np.dot`, NumPy performs a sum-product over the last axis of `A` and the second-to-last axis of `B`, but it results in a 4D output because it combines all other dimensions.

In [ ]:

```
dot_res = np.dot(A, B)
print(f"Result shape of np.dot(A, B): {dot_res.shape}")
```


▶ Run in Colab

3. Using the `@` Operator (Matmul)

The `@` operator treats the leading dimension as a batch. It performs matrix multiplication on the last two dimensions for each item in the batch, preserving the 3D structure.

In [ ]:

```
matmul_res = A @ B
print(f"Result shape of A @ B:         {matmul_res.shape}")
print("\nObservation: @ preserved the batch size of 2, while np.dot expanded the dimensions.")
```


▶ Run in Colab

Performance Comparison: `np.dot` vs `@`

In addition to dimensional behavior, it's useful to know if one is faster than the other. We'll use the `%timeit` magic command to compare their execution speeds on large 2D matrices.

In [ ]:

```
import numpy as np\n\n# Create large 2D matrices for benchmarking\nsize = 1000\nX = np.random.rand(size, size)\nY = np.random.rand(
```


Run in Colab

NumPy Attributes: Shape and Dimensions

Every NumPy array has attributes that allow you to inspect its structure:

1. `.ndim`: Returns the number of dimensions (axes).
2. `.shape`: Returns a tuple showing the length of each dimension.
3. `.size`: Returns the total number of elements in the array.

In [ ]:

```python
import numpy as np

# Create different types of arrays
scalar = np.array(10)
vector = np.array([1, 2, 3])
matrix = np.array([[1, 2, 3], [4, 5, 6]])
tensor = np.random.rand(2, 3, 4)

def inspect_array(name, arr):
    print(f"--- {name} ---")
    print(f"Dimensions (ndim): {arr.ndim}")
    print(f"Shape:             {arr.shape}")
```


▶ Run in Colab

NumPy Universal Functions (ufuncs)

A **universal function** (or **ufunc**) is a function that operates on `ndarrays` in an element-by-element fashion.

Why are ufuncs better than Python loops?

1. **Vectorization**: ufuncs perform operations across entire arrays at once, removing the need for slow Python `for` loops.
2. **C-Speed**: The actual calculation happens in optimized C code, which is significantly faster than the Python interpreter.
3. **Hardware Optimization**: They can take advantage of SIMD (Single Instruction, Multiple Data) on your CPU to process multiple numbers in a single clock cycle.

Example 1: Basic Arithmetic ufuncs

Instead of looping, we use operators like `+`, `-`, `*`, or functions like `np.add()` directly on the arrays.

In [ ]:

```python
import numpy as np

arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([10, 20, 30, 40])

# np.add(arr1, arr2) is the ufunc behind 'arr1 + arr2'
sum_result = np.add(arr1, arr2)

print(f"Addition result: {sum_result}")
```


▶ Run in Colab

Example 2: Mathematical Transformations

NumPy provides ufuncs for almost every mathematical operation, such as `np.sin`, `np.exp`, and `np.log`.

In [ ]:

```
angles = np.array([0, np.pi/2, np.pi])

# Applying sine to the entire array at once
sine_values = np.sin(angles)

print(f"Sine values: {sine_values}")
```


Run in Colab

Example 3: Comparison ufuncs

You can compare two arrays element-wise to create a boolean mask, which is a key technique in data filtering.

In [ ]:

```
data = np.array([10, 5, 20, 15, 30])

# Use the 'greater than' ufunc
mask = data > 15

print(f"Data: {data}")
print(f"Mask (Values > 15): {mask}")
```


▶ Run in Colab

Performance Comparison: Python Loop vs. NumPy ufunc

To see the advantage of vectorization, we will benchmark a simple task: adding 1 to every element in a large array (1 million elements).

In [ ]:

```python
import numpy as np

# Create an array with 1 million elements for benchmarking
size = 1000000
arr = np.arange(size)
print(f"Array initialized with {size:,} elements.")
```


▶ Run in Colab

NumPy Aggregations in Matrices

Aggregations are functions that summarize a large dataset into a single value or a smaller set of values. In NumPy, these functions are highly optimized to process matrices efficiently.

1. Calculating the Sum

The `np.sum()` function calculates the total of all elements in the matrix. This is often used in AI to calculate total error or loss across a batch of data.

In [ ]:

```python
import numpy as np

matrix = np.array([[1, 2, 3], [4, 5, 6]])

# Total sum of all elements
total_sum = np.sum(matrix)

print("Matrix:")
print(matrix)
print(f"\nTotal Sum: {total_sum}")
```


▶ Run in Colab

2. Calculating the Mean (Average)

The `np.mean()` function computes the arithmetic mean. In machine learning, this is fundamental for normalizing data or finding the average performance of a model.

In [ ]:

```
# Average of all elements
average = np.mean(matrix)

print(f"Matrix Mean: {average}")
```


Run in Colab

3. Finding Minimum and Maximum Values

`np.min()` and `np.max()` allow you to find the smallest and largest values in your dataset, which is useful for identifying outliers or scaling features between 0 and 1.

In [ ]:

```
min_val = np.min(matrix)
max_val = np.max(matrix)

print(f"Minimum Value: {min_val}")
print(f"Maximum Value: {max_val}")
```


▶ Run in Colab

4. Aggregations Across Axes

One of the most powerful features of NumPy is aggregating along specific dimensions using the `axis` parameter:

- **`axis=0`**: Aggregates vertically (per column).
- **`axis=1`**: Aggregates horizontally (per row).

In [ ]:

```
# Sum along columns (axis 0)
col_sum = np.sum(matrix, axis=0)

# Sum along rows (axis 1)
row_sum = np.sum(matrix, axis=1)

print(f"Column Sums: {col_sum}")
print(f"Row Sums: {row_sum}")
```


▶ Run in Colab

`np.mean` vs `np.average`

In many cases, these two functions return the same result. However, they have one key difference:

- `np.mean`: Always computes the standard arithmetic mean.
- `np.average`: Can compute a **weighted average** if you provide a `weights` parameter.

Weighted averages are crucial in AI when some data points (like recent observations or more reliable sensors) should influence the result more than others.

In [ ]:

```python
import numpy as np

data = np.array([10, 20, 30])
weights = np.array([0.1, 0.2, 0.7]) # The last number has the highest influence

# Standard mean
standard_mean = np.mean(data)

# Weighted average
weighted_avg = np.average(data, weights=weights)

print(f"Data: {data}")
```


▶ Run in Colab

Visualizing Shape

Think of the shape as a 'blueprint' for the data:

- `shape (3,)`: 1D array with 3 elements.
- `shape (2, 3)`: 2D array with 2 rows and 3 columns.
- `shape (2, 3, 4)`: 3D array consisting of 2 matrices, each being 3×4.

Reshaping NumPy Arrays

Reshaping means changing the shape of an array. The shape of an array is the number of elements in each dimension. By reshaping, we can add or remove dimensions or change the number of elements in each dimension.

The Golden Rule of Reshaping

To reshape an array, the **total number of elements must remain the same**. For example, an array with 8 elements can be reshaped into `(4, 2)`, `(2, 4)`, `(2, 2, 2)`, or `(8, 1)`, but not `(3, 3)` (which requires 9 elements).

In [ ]:

```python
import numpy as np

# Create a 1D array with 12 elements
arr = np.arange(12)
print(f"Original 1D  {arr}")
print(f"Original Shape: {arr.shape}\n")

# Reshape to 2D (3 rows, 4 columns)
arr_2d = arr.reshape(3, 4)
print("to 2D (3, 4):")
print(arr_2d)
```


▶ Run in Colab

Using -1 as an Unknown Dimension

NumPy allows you to specify one dimension as `-1`. This tells NumPy to automatically calculate the correct size for that dimension based on the total number of elements. This is very helpful when you know one dimension (like batch size) but don't want to do the math for the others.

In [ ]:

```
# We have 12 elements. Let's say we want 2 rows and let NumPy figure out the columns.
auto_reshaped = arr.reshape(2, -1)

print(f"Shape with -1: {auto_reshaped.shape}")
print(auto_reshaped)
```


▶ Run in Colab

Flattening Arrays

Flattening is the process of converting a multi-dimensional array back into a 1D array. This is a common step in Neural Networks when transitioning from Convolutional layers to Fully Connected (Dense) layers.

In [ ]:

```python
# Flatten the 2D array back to 1D
flattened = arr_2d.reshape(-1)

print(f"Flattened shape: {flattened.shape}")
print(f"Flattened array: {flattened}")
```


Run in Colab

Popular Matrix Operations

In linear algebra, matrices are more than just grids of numbers; they are operators. Here are the most frequent operations used in AI data pipelines.

1. Matrix Addition and Subtraction

Addition and subtraction are performed element-wise. Both matrices must have the exact same dimensions.

In [ ]:

```python
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Element-wise addition
matrix_sum = A + B

print("Matrix A + B:")
print(matrix_sum)
```


▶ Run in Colab

2. Matrix Multiplication (Dot Product)

This is not element-wise. The number of columns in the first matrix must match the number of rows in the second. In AI, this represents passing data through a layer of 'weights'.

In [ ]:

```
# Using the @ operator for cleaner matrix multiplication
matrix_product = A @ B

print("Matrix A dot B:")
print(matrix_product)
```


▶ Run in Colab

3. Matrix Transposition

Transposing a matrix means switching its rows and columns. This is common when you need to align dimensions for multiplication.

In [ ]:

```
matrix_c = np.array([[1, 2, 3], [4, 5, 6]])

# Transpose using the .T attribute
transposed = matrix_c.T

print(f"Original: {matrix_c.shape}")
print(matrix_c)
print(f"\nTransposed: {transposed.shape}")
print(transposed)
```


Run in Colab

4. Matrix Inverse

The inverse of a matrix A is A^{-1} such that $A \cdot A^{-1} = I$ (the Identity Matrix). This is useful for solving equations, though in AI we often use 'pseudo-inverses' for stability.

In [ ]:

```
from numpy.linalg import inv

# Create a square matrix
square_matrix = np.array([[1, 2], [3, 4]])

# Calculate the inverse
inverse_matrix = inv(square_matrix)

print("Inverse of Matrix:")
print(inverse_matrix)

# Verify: A @ inv(A) should be Identity (1s on diagonal)
```


▶ Run in Colab

Understanding Matrix Multiplication (Dot Product)

Matrix multiplication is a way of combining two matrices to produce a third. In AI, this is often used to apply 'weights' to input data.

The Rule of Dimensions: To multiply two matrices, the number of **columns** in the first matrix must equal the number of **rows** in the second.

- If Matrix A is $(m \times n)$ and Matrix B is $(n \times p)$:
- The result will be a matrix of shape $(m \times p)$.

In [ ]:

```python
import numpy as np

# A (2x3 matrix)
A = np.array([[1, 2, 3],
              [4, 5, 6]])

# B (3x2 matrix)
B = np.array([[7, 8],
              [9, 10],
              [11, 12]])

# Multiplying A (2x3) @ B (3x2) results in a (2x2) matrix
```


▶ Run in Colab

NumPy Indexing and Slicing

Indexing allows you to access individual elements or specific sections of an array. Like Python lists, NumPy uses **zero-based indexing** (the first element is at index 0).

1. Basic Indexing (1D and 2D)

For 1D arrays, you use a single index. For 2D matrices, you use a comma-separated tuple of `[row, column]`.

In [ ]:

```python
import numpy as np

arr_1d = np.array([10, 20, 30, 40])
matrix_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(f"1D: Second element: {arr_1d[1]}")
print(f"2D: Element at row 0, col 2: {matrix_2d[0, 2]}")
print(f"2D: Entire second row: {matrix_2d[1]}")
```


▶ Run in Colab

2. Slicing Arrays

Slicing uses the `[start:stop:step]` syntax. If you omit the start, it begins at 0. If you omit the stop, it goes to the end.

In [ ]:

```
# Get elements from index 1 to 3
slice_1d = arr_1d[1:4]

# Get the first two columns of the first two rows
slice_2d = matrix_2d[:2, :2]

print(f"1D Slice: {slice_1d}")
print("2D Top-Left Sub-matrix:")
print(slice_2d)
```


▶ Run in Colab

3. Boolean Indexing (Data Filtering)

This is a powerful 'masking' technique. You pass a condition into the brackets, and NumPy returns only the elements where the condition is `True`.

In the following example, `data > 10` creates a 'Boolean Mask' (a list of True/False values). When we apply this mask back to the original array, NumPy only selects the values corresponding to `True`.

In [ ]:

```
data = np.array([5, 12, 8, 20, 3, 15])

filtered_data = data[data > 10]

print(f"Original Data: {data}")
print(f"Values > 10:   {filtered_data}")
```


▶ Run in Colab

4. Selecting Specific Rows and Columns (2D)

In NumPy, you can select specific rows and columns by using a comma-separated syntax: `array[row_selection, column_selection]`.

- **Selecting a Row:** Use a single integer for the row and `:` for all columns.
- **Selecting a Column:** Use `:` for all rows and a single integer for the column.
- **Selecting Multiple Specific Indices:** You can pass lists of indices to pick non-contiguous rows or columns.

4. Selecting Specific Rows and Columns (2D)

First, we create a 4×4 matrix to demonstrate various selection techniques.

In [ ]:

```python
import numpy as np

# Create a 4x4 matrix
matrix = np.array([
    [10, 11, 12, 13],
    [20, 21, 22, 23],
    [30, 31, 32, 33],
    [40, 41, 42, 43]
])

print("Original Matrix:")
print(matrix)
```


Run in Colab

A. Selecting a Single Row

To select an entire row, specify the row index and use the colon `:` to indicate that you want all columns for that row.

In [ ]:

```
row_1 = matrix[1, :]
print(f"Second Row (index 1): {row_1}")
```


▶ Run in Colab

B. Selecting a Single Column

Similarly, to select a column, use the colon `:` for the row position and specify the index for the column.

In [ ]:

```
col_2 = matrix[:, 2]
print(f"Third Column (index 2): {col_2}")
```


▶ Run in Colab

C. Selecting Non-Contiguous Rows

You can pass a list of integers to select specific, non-adjacent rows or columns.

In [ ]:

```
specific_rows = matrix[[0, 3], :]
print("First and Fourth Rows:")
print(specific_rows)
```


▶ Run in Colab

D. Extracting a Sub-Matrix

By combining slices for both rows and columns, you can extract a specific sub-grid from the larger matrix.

In [ ]:

```
sub_matrix = matrix[1:3, 1:3]
print("Middle 2x2 Sub-matrix:")
print(sub_matrix)
```


Run in Colab

In [ ]:

```python
import numpy as np

# Create a 4x4 matrix
matrix = np.array([
    [10, 11, 12, 13],
    [20, 21, 22, 23],
    [30, 31, 32, 33],
    [40, 41, 42, 43]
])

print("Original Matrix:")
print(matrix)
```


Run in Colab

1. Selecting a Single Row

To extract an entire row, we specify the row index and use a colon (`:`) for the column index, which signifies 'all columns' in that specific row.

In [ ]:

```
row_1 = matrix[1, :]
print(f"Second Row (index 1): {row_1}")
```


Run in Colab

2. Selecting a Single Column

To extract a vertical column, we use the colon (`:`) in the row position and provide a specific index for the column.

In [ ]:

```
col_2 = matrix[:, 2]
print(f"Third Column (index 2): {col_2}")
```


Run in Colab

3. Selecting Non-Contiguous Rows

By passing a list of integers as the row selection, we can pick specific rows that are not next to each other.

In [ ]:

```
specific_rows = matrix[[0, 3], :]
print("First and Fourth Rows:")
print(specific_rows)
```


▶ Run in Colab

4. Extracting a Sub-matrix

We can extract a smaller grid from the larger matrix by providing range slices for both rows and columns.

In [ ]:

```
sub_matrix = matrix[1:3, 1:3]
print("Middle 2x2 Sub-matrix:")
print(sub_matrix)
```


Run in Colab

Coordinate Systems and Geometry

This section covers fundamental geometric concepts used to represent and calculate relationships between points in space, which is critical for computer vision and spatial data analysis.

1. Cartesian Coordinates

Concept: The Cartesian coordinate system specifies each point uniquely in a plane by a set of numerical coordinates. In a 2D system, these are the x (horizontal) and y (vertical) distances from a fixed origin $(0,0)$.

NumPy Implementation: In NumPy, we represent coordinates as vectors or arrays. A single point is a 1D array, while a collection of points is often represented as a 2D matrix where each row is a coordinate pair.

In [ ]:

```python
import numpy as np

# Representing a point (x, y)
point = np.array([3, 4])

# Representing multiple points
points = np.array([
    [0, 0],   # Origin
    [3, 4],
    [6, 8]
])
```


Run in Colab

2. Distance Between Points (Euclidean Distance)

Concept: The Euclidean distance is the 'straight-line' distance between two points (x_1, y_1) and (x_2, y_2). The formula is $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$.

NumPy Implementation: We can calculate this using element-wise subtraction and the `np.linalg.norm` function, which computes the magnitude of the vector difference.

In [ ]:

```
p1 = np.array([1, 2])
p2 = np.array([4, 6])

# Method 1: Manual calculation
distance_manual = np.sqrt(np.sum((p2 - p1)**2))

# Method 2: Using linalg.norm (standard approach)
distance_norm = np.linalg.norm(p2 - p1)

print(f"Point 1: {p1}, Point 2: {p2}")
print(f"Euclidean Distance: {distance_norm:.2f}")
```


▶ Run in Colab

3. Midpoint

Concept: The midpoint is the exact center between two points. It is calculated by taking the average of the x coordinates and the average of the y coordinates: $(\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$.

NumPy Implementation: Since NumPy operations are vectorized, we can simply add the two point arrays and divide by 2.

In [ ]:

```
p1 = np.array([2, 10])
p2 = np.array([8, 4])

# Vectorized average
midpoint = (p1 + p2) / 2

print(f"Midpoint of {p1} and {p2} is: {midpoint}")
```


▶ Run in Colab

4. Pythagorean Theorem

Concept: In a right-angled triangle, the square of the hypotenuse (c) is equal to the sum of the squares of the other two sides (a and b): $a^2 + b^2 = c^2$.

NumPy Implementation: We can use `np.hypot(a, b)` to calculate the hypotenuse directly, which is more numerically stable than manual squaring and square-rooting.

In [ ]:

```
side_a = 3
side_b = 4

# Calculate hypotenuse c
hypotenuse = np.hypot(side_a, side_b)

print(f"Side A: {side_a}, Side B: {side_b}")
print(f"Hypotenuse (C): {hypotenuse}")
```


Run in Colab

5. Area of a Triangle (Shoelace Formula)

Concept: If you know the coordinates of the three vertices of a triangle (x_1, y_1), (x_2, y_2), and (x_3, y_3), you can calculate the area using the Shoelace Formula:

$$\text{Area} = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

NumPy Implementation: We can use array indexing to plug these values directly into the formula.

Step 1: Define the Vertices

We define the three points of the triangle as 1D NumPy arrays. In this example, we'll use a right-angled triangle with vertices at $(0,0)$, $(4,0)$, and $(0,3)$ to easily verify our result ($Area = 6$).

In [ ]:

```python
import numpy as np

# Define vertices as [x, y] coordinates
A = np.array([0, 0])
B = np.array([4, 0])
C = np.array([0, 3])

print(f"Vertex A: {A}")
print(f"Vertex B: {B}")
print(f"Vertex C: {C}")
```


▶ Run in Colab

Step 2: Implement the Shoelace Logic

The Shoelace Formula calculates the area by performing cross-multiplication between the coordinates of the vertices. The formula for the area of a triangle with vertices $A(x_1, y_1)$, $B(x_2, y_2)$, and $C(x_3, y_3)$ is:

$$\text{Area} = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

We will use NumPy indexing to extract specific coordinates from our arrays to build this expression.

In [ ]:

```
# Accessing coordinates: array[0] is x, array[1] is y
term1 = A[0] * (B[1] - C[1])
term2 = B[0] * (C[1] - A[1])
term3 = C[0] * (A[1] - B[1])

# Sum the terms
shoelace_sum = term1 + term2 + term3
print(f"Sum of products: {shoelace_sum}")
```


Run in Colab

Step 3: Apply Absolute Value and Halve

The final step is to take the absolute value of our sum (to ensure the area is positive) and multiply by 0.5.

In [ ]:

```
# Calculate final area using np.abs()
area = 0.5 * np.abs(shoelace_sum)

print(f"Calculated Area: {area}")
print(f"Expected Area (0.5 * base * height): {0.5 * 4 * 3}")
```


▶ Run in Colab

1. Functions

Mathematical Concept: A function is a relationship between a set of inputs and a set of possible outputs where each input is related to exactly one output. In mathematical notation, this is typically written as $f(x) = y$. In data science, we often treat functions as 'black boxes' that transform raw data into useful predictions.

Code Explanation: In the following NumPy implementation, we define a standard Python function that leverages NumPy's vectorized operations. This allows the function to process a single number (scalar) or a whole collection of numbers (array) simultaneously without modification.

In [ ]:

```python
import numpy as np

def simple_function(x):
    # This function doubles the input
    return 2 * x

# Testing with a NumPy array
inputs = np.array([1, 2, 3, 4, 5])
outputs = simple_function(inputs)

print(f"Inputs:  {inputs}")
print(f"Outputs: {outputs}")
```


▶ Run in Colab

2. Inputs and Outputs

Mathematical Concept: The **Input** (Domain) is the value you provide to the function, while the **Output** (Range) is the result produced after the function's logic is applied. In AI, inputs are often referred to as 'Features' and outputs are referred to as 'Targets' or 'Labels'.

Code Explanation: We use `np.linspace` to generate a structured range of inputs. This is a common technique used to create smooth visualizations of how a function behaves across a specific interval.

In [ ]:

```
# Generate 5 inputs between 0 and 10
feature_inputs = np.linspace(0, 10, 5)

# Define a function to calculate a range of outputs
def process_data(data):
    return data / 10

target_outputs = process_data(feature_inputs)

print(f"Feature Inputs (Domain): {feature_inputs}")
print(f"Target Outputs (Range):  {target_outputs}")
```


▶ Run in Colab

3. Linear Functions

Mathematical Concept: A linear function is a function that creates a straight line when plotted. It follows the formula $y = mx + b$, where m is the **slope** (how steep the line is) and b is the **y-intercept** (where the line crosses the vertical axis). In Machine Learning, this is the basis for Linear Regression.

Code Explanation: The code below implements $y = 3x + 2$. By using NumPy arrays, we calculate the y coordinates for multiple x points in a single vectorized step.

In [ ]:

```
x_coords = np.array([0, 1, 2, 3, 4])

slope = 3
intercept = 2

# Calculate y = mx + b
y_coords = (slope * x_coords) + intercept

print(f"X coordinates: {x_coords}")
print(f"Y coordinates (3x + 2): {y_coords}")
```


▶ Run in Colab

4. Quadratic Functions

Mathematical Concept: A quadratic function is a polynomial function of degree 2, following the general form $y = ax^2 + bx + c$. When graphed, it forms a curve called a **parabola**. Quadratic functions are essential in AI for calculating 'Loss Functions' (like Mean Squared Error), which are often parabolic.

Code Explanation: We implement $y = x^2$. We use the `**` operator, which NumPy applies element-wise to every number in the input array, creating the characteristic curved output values.

In [ ]:

```
x_values = np.arange(-3, 4) # [-3, -2, -1, 0, 1, 2, 3]

# Quadratic logic: y = x^2
y_values = x_values ** 2

print(f"X values: {x_values}")
print(f"Y values (x squared): {y_values}")
```


▶ Run in Colab

Visualizing the Parabola

To better understand quadratic functions, we can plot them. A parabola is symmetrical, and for $y = x^2$, the lowest point (the vertex) is at $(0,0)$. This shape is vital in optimization, as machine learning models try to 'roll down' this curve to find the minimum error.

In [ ]:

```
#### Step 1: Data Preparation
To create a smooth curve for the parabola, we generate a high-density range of 100 values between -10 and 10 using `np.linspace`. W
```


▶ Run in Colab

In [ ]:

```
import matplotlib.pyplot as plt
import numpy as np

x_plot = np.linspace(-10, 10, 100)
y_plot = x_plot**2
```


▶ Run in Colab

Step 2: Basic Plotting

We initialize a figure and plot the x and y coordinates. We use a red line with a specific width and add a LaTeX-formatted label for the legend.

In [ ]:

```
plt.figure(figsize=(8, 5))
plt.plot(x_plot, y_plot, label='$y = x^2$', color='red', linewidth=2)
```


Run in Colab

Step 3: Styling and Display

To make the graph readable, we add a title, axis labels, and a grid. We also draw the black origin lines (axes) and display the legend before showing the final result.

In [ ]:

```
plt.title('Visualization of a Quadratic Function (Parabola)')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True, linestyle='--', alpha=0.6)
plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)
plt.legend()
plt.show()
```


▶ Run in Colab

In [ ]:

```
import matplotlib.pyplot as plt
import numpy as np

def linear_func(x):
    return 2 * x + 10

def quadratic_func(x):
    return x**2
```


▶ Run in Colab

Step 1: Defining the Mathematical Functions

We start by defining two functions: a linear function $y = 2x + 10$ and our standard quadratic function $y = x^2$. This allows us to reuse the logic easily during the plotting phase.

In [ ]:

```
x = np.linspace(-10, 10, 100)
y_lin = linear_func(x)
y_quad = quadratic_func(x)
```


Run in Colab

Step 2: Data Generation

We use `np.linspace` to create 100 points between -10 and 10. We then calculate the corresponding y values for both the linear and quadratic models using the functions defined in the previous step.

In [ ]:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Subplot 1: Linear Function
ax1.plot(x, y_lin, color='blue', label='$y = 2x + 10$')
ax1.set_title('Linear Function (1st Degree)')
ax1.grid(True, alpha=0.3)
ax1.legend()

# Subplot 2: Quadratic Function
ax2.plot(x, y_quad, color='red', label='$y = x^2$')
ax2.set_title('Quadratic Function (2nd Degree)')
ax2.grid(True, alpha=0.3)
```


▶ Run in Colab

Step 3: Creating Subplots

Using `plt.subplots(1, 2)`, we create a side-by-side comparison. The first subplot (`ax1`) displays the linear relationship, characterized by a constant slope, while the second (`ax2`) shows the parabolic curve of the quadratic function.

In [ ]:

```
plt.tight_layout()
plt.show()
```


Run in Colab

Understanding the Comparison

1. **Linear ($y = mx + b$):** The blue line represents a constant rate of change. As x increases, y increases at a fixed ratio. In machine learning, this represents the simplest relationship between variables.
2. **Quadratic ($y = ax^2$):** The red curve shows that the rate of change is not constant; it accelerates as we move away from the origin. This creates the 'U-shape' known as a parabola.
3. **Intersection:** Notice that the linear and quadratic functions cross at specific points where $x^2 = 2x + 10$. Visualizing them side-by-side helps us identify which function grows faster at different intervals of x.

Interactive Coefficient Exploration

In this section, we will study how changing specific numbers (coefficients) in an equation transforms its visual representation. Understanding these shifts is a core part of 'tuning' models in Artificial Intelligence.

In [ ]:

```
def plot_linear_variation(m_values, b):
    x = np.linspace(-10, 10, 100)
    plt.figure(figsize=(10, 6))

    for m in m_values:
        y = m * x + b
        plt.plot(x, y, label=f'm={m} (y={m}x+{b})')

    plt.axhline(0, color='black', lw=1)
    plt.axvline(0, color='black', lw=1)
    plt.title("Effect of Slope (m) on Linear Functions")
    plt.legend()
```


▶ Run in Colab

1. Linear Coefficients: Slope (m)

The coefficient m represents the **slope**.

- **Higher m**: The line becomes steeper.
- **Lower m (close to 0)**: The line becomes flatter.
- **Negative m**: The line slopes downwards from left to right.

Detailed Breakdown of Slope (m)

In the linear equation $y = mx + b$, the slope **m** determines the rate of change of the function. Here is how it affects the line:

- **Positive Slope ($m > 0$):** The line rises from left to right. As x increases, y also increases.
- **Negative Slope ($m < 0$):** The line falls from left to right. As x increases, y decreases.
- **Zero Slope ($m = 0$):** The line is perfectly horizontal ($y = b$). There is no change in y regardless of x.
- **Magnitude ($|m|$):** The absolute value of m determines steepness. A value of 2 is steeper than 0.5. In machine learning, this slope often represents the 'weight' or importance of a specific feature.

In [ ]:

```
def plot_quadratic_variation(a_values):
    x = np.linspace(-5, 5, 100)
    plt.figure(figsize=(10, 6))

    for a in a_values:
        y = a * (x**2)
        plt.plot(x, y, label=f'a={a} (y={a}x²)')

    plt.axhline(0, color='black', lw=1)
    plt.axvline(0, color='black', lw=1)
    plt.title("Effect of Coefficient (a) on Parabolas")
    plt.legend()
```


▶ Run in Colab

2. Quadratic Coefficients: The Square Term (a)

The coefficient a in $y = ax^2$ controls the 'steepness' of the curve:

- **Large a**: The parabola becomes narrower and 'shoots up' faster.
- **Small a**: The parabola becomes wider and flatter.
- **Negative a**: The parabola flips upside down (concave down).

Mini-Project: Real Estate Price Prediction

In machine learning, the slope (m) isn't just a number—it's a **weight**. Let's say we are predicting house prices based on square footage.

The Scenario:

- **x**: Size of the house (Square Feet).
- **y**: Price of the house.
- **Slope (m)**: The cost per square foot.

By comparing different slopes, we can see how 'steepness' represents different market conditions (e.g., Luxury vs. Affordable housing).

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate House Sizes (from 500 to 3500 sq ft)
house_sizes = np.linspace(500, 3500, 50)

# 2. Define different 'Price per Square Foot' (Slopes)
low_end_m = 100    # $100 per sq ft
mid_range_m = 250   # $250 per sq ft
luxury_m = 600      # $600 per sq ft

# 3. Calculate Prices (y = mx)
```


Run in Colab

What this graph tells us:

1. **High Steepness ($m=600$):** In the luxury market, every extra square foot adds a lot of money to the price. The model is very 'sensitive' to the size.
2. **Low Steepness ($m=100$):** In the affordable market, the price increases slowly as size increases. The 'weight' of the size feature is much lower.

In AI terms: When training a model, the algorithm is trying to find the exact **steepness** that best fits the historical sales data!

Part 3: Data Analysis with Pandas

What is Pandas?

Pandas is a powerful, open-source Python library used for **data manipulation and analysis**. Think of it as the ultimate Swiss Army knife for data. While NumPy is great for math and multi-dimensional arrays, Pandas is designed to handle **tabular data**—the kind of data you see in spreadsheets or SQL tables.

Why is it so popular?

- **Flexibility:** It can handle missing data, different data types in the same table, and complex filtering with ease.
- **Integration:** It works perfectly with NumPy, Matplotlib, and Scikit-learn.
- **Speed:** It is built on top of NumPy, making it incredibly fast even for large datasets.

Pandas vs. Excel

If you have ever used Microsoft Excel or Google Sheets, you already understand the basics of Pandas. However, there are some key differences that make Pandas better for AI and large-scale data work:

Feature	Excel	Pandas
Data Size	Struggles with millions of rows.	Can handle millions of rows efficiently.
Automation	Requires manual clicks or complex VBA macros.	Easily automated with Python scripts.
Reproducibility	Hard to track exactly what changes were made.	Every cleaning step is recorded in your code.
Integration	Difficult to feed directly into AI models.	Designed to be the first step in an AI pipeline.

The Excel Analogy

To help you visualize how Pandas works, let's use an analogy of a physical workbook:

1. **The Excel File (.xlsx) = The DataFrame:** In Pandas, your entire table is called a **DataFrame**. Just like an Excel file, it has rows and columns.
2. **A Single Column = A Series:** In Excel, you might have a 'Price' column. In Pandas, a single column is called a **Series**. A DataFrame is essentially just a collection of Series objects aligned by an index.
3. **The Row Numbers = The Index:** In Excel, you see numbers on the left (1, 2, 3...). Pandas uses an **Index** to keep track of rows, but unlike Excel, the index can be names, dates, or ID numbers.
4. **Formulas = Vectorized Operations:** Instead of dragging a formula down a column like in Excel, you apply a calculation to the whole Series at once in Pandas.

Pandas Basics: The Building Blocks of Pandas

1. The Pandas Series

A **Series** is the simplest data structure in Pandas. Think of it as a **single column** in a spreadsheet or a specialized Python list.

Key Characteristics:

- **One-Dimensional:** It only grows in one direction (down).
- **Labeled:** Every item in a Series has a label called an **Index** (usually starting from 0).
- **Homogeneous:** Usually, all the data in a single Series is of the same type (like all integers or all strings).

Example: A list of prices or a list of names is a Series.

2. The Pandas DataFrame

A **DataFrame** is the most commonly used structure. If a Series is a single column, a DataFrame is the **entire table**.

Key Characteristics:

- **Two-Dimensional:** It has both rows and columns (like an Excel sheet).
- **Heterogeneous:** Different columns can hold different types of data (e.g., one column for names, one for ages, and one for prices).
- **Alignment:** It keeps all your data aligned. If you look at row #5, you'll see the name, age, and price for the exact same person.

Analogy: If a Series is a single 'bead string,' a DataFrame is a 'fabric' woven from many strings.

Creating a Pandas Series

To create a Series, we use the `pd.Series()` constructor. In this first example, we will convert a simple Python list of integers into a Pandas Series. Notice how Pandas automatically adds a numeric index starting from 0.

In [ ]:

```python
import pandas as pd

# A list of data points
data = [10, 20, 30, 40]

# Converting the list to a Series
my_series = pd.Series(data)

print(my_series)
```


▶ Run in Colab

Customizing the Index

One of the most powerful features of a Series is that you can define your own labels for the index. This makes the data much easier to look up later, similar to how you might use a key in a dictionary.

In [ ]:

```python
# Creating a Series with custom index labels
import pandas as pd
prices = [1.50, 2.00, 3.50]
labels = ["Apple", "Banana", "Cherry"]

fruit_prices = pd.Series(data=prices, index=labels)

print(fruit_prices)
```


▶ Run in Colab

Breakdown of the Code:

- `import pandas as pd`: We bring in the Pandas library and give it the nickname `pd` so we don't have to type the full word every time.
- `prices = [...]`: This is a standard Python list containing the numeric values we want to store.
- `labels = [...]`: This is another list containing the names (labels) we want to associate with those prices.
- `pd.Series(data=prices, index=labels)`: This is the 'constructor.' We tell Pandas to take our `prices` and use the `labels` as the custom index for each price.
- `fruit_prices`: This is the variable name we chose to store our newly created Series object.

Accessing Data by Label

Once you have a custom index, you can access specific values using the label name inside square brackets. This is much more intuitive than remembering a row number.

In [ ]:

```
# Accessing a specific value using its index label
banana_price = fruit_prices["Banana"]

print(f"The price of a Banana is: {banana_price}")
```


Run in Colab

Loading CSV Files in Pandas

Most data in AI projects comes from external files, with **CSV (Comma Separated Values)** being the most common format. To work with these files in Google Colab, we first need to ensure the file is accessible to our environment.

1. Uploading a File to Colab

If your file is on your local computer, you can upload it directly to the Colab session using the `files` module from `google.colab`. When you run the following code, a button will appear allowing you to select a file from your hard drive.

In [ ]:

```
from google.colab import files
uploaded = files.upload()
```


Run in Colab

2. Reading a CSV into a DataFrame

Once the file is uploaded (you can see it in the folder icon on the left sidebar), we use the `pd.read_csv()` function. This function reads the file and automatically converts it into a tabular **DataFrame** object.

In [ ]:

```
import pandas as pd

df = pd.read_csv('your_filename.csv')

display(df.head())
```


▶ Run in Colab

3. Understanding `read_csv` Parameters

Sometimes CSV files use different formats. You can customize how Pandas reads the file using parameters:

- **`sep`**: Change the delimiter (e.g., `sep='\t'` for tab-separated files).
- **`header`**: Specify which row contains the column names.
- **`usecols`**: Load only a specific subset of columns to save memory.

In [ ]:

```
df_custom = pd.read_csv('your_filename.csv', sep=',', header=0)

print(df_custom.shape)
```


Run in Colab

Inspecting DataFrame Structure

Once a dataset is loaded, the first step is often to check its structure. Pandas provides built-in methods to view column names, data types, and memory usage.

1. Using the info() Method

The `info()` method provides a comprehensive summary of the DataFrame. It shows the number of rows (entries), the names of all columns, the number of non-null values in each column, and the data types.

In [ ]:

```
df.info()
```


Run in Colab

2. Checking Data Types with dtypes

If you specifically need to see the data types (e.g., int64, float64, object) for each column without the extra summary information, you can use the `dtypes` attribute.

In [ ]:

```
print(df.dtypes)
```


Run in Colab

3. Summary Statistics with describe()

To get a quick overview of the distribution of your numeric data, use `describe()`. It calculates count, mean, standard deviation, minimum, maximum, and percentiles for every numeric column.

In [ ]:

```
display(df.describe())
```


Run in Colab

Deep Dive: Understanding `df.info()`

The `info()` method is your first line of defense when exploring a new dataset. It provides a technical summary that helps you identify potential issues before you start your analysis.

What `info()` tells you:

- **RangeIndex:** The total number of rows (entries).
- **Data Columns:** The total number of columns.
- **Non-Null Count:** How many values in each column are *not* missing. If this number is lower than the total number of entries, you have missing data (NaNs) to handle.
- **Dtype:** The data type assigned to the column (e.g., `int64` for whole numbers, `float64` for decimals, or `object` for text).
- **Memory Usage:** How much RAM the DataFrame is consuming. This is vital when working with massive AI datasets.

In [ ]:

```
df.info()
```


Run in Colab

Selecting and Filtering Data in Pandas

Selecting specific subsets of your data is a core task in data analysis. Pandas provides three primary ways to access data:

1. **Column Selection:** Accessing one or more columns by name.
2. **Label-based Selection (`loc`):** Selecting rows and columns by their specific labels/names.
3. **Position-based Selection (`iloc`):** Selecting rows and columns by their integer index position (0, 1, 2...).
4. **Boolean Indexing:** Filtering rows based on a logical condition (e.g., all rows where 'Age' > 25).

1. Selecting Columns

You can select a single column using square brackets with the column name. To select multiple columns, you pass a **list** of column names inside the square brackets.

In [ ]:

```
single_col = df['column_name']
multiple_cols = df[['col_a', 'col_b']]
display(multiple_cols.head())
```


▶ Run in Colab

2. Label-based Selection with `.loc`

The `.loc` attribute is used to select data using labels. The syntax is `df.loc[row_label, column_label]`. You can use a colon `:` to select all items in a dimension.

In [ ]:

```
row_subset = df.loc[0:5, 'column_name']
display(row_subset)
```


Run in Colab

3. Position-based Selection with `.iloc`

The `.iloc` attribute works exactly like standard Python list slicing. It ignores the names of your rows or columns and looks strictly at their numeric position.

In [ ]:

```
first_five_rows_three_cols = df.iloc[0:5, 0:3]
display(first_five_rows_three_cols)
```


Run in Colab

Beginner's Guide to `.iloc` selection

In the code above, `df.iloc[0:5, 0:3]` is a way to tell Pandas exactly which 'box' of data you want to grab based on its location, not its name.

- **The 'i' in .iloc**: Stands for **Integer**. This means you must always use whole numbers (0, 1, 2...) inside the brackets.
- **The First Part (`0:5`)**: This refers to the **rows**. Just like standard Python lists, `0:5` means 'start at row 0 and go up to, but do not include, row 5'.
- **The Second Part (`0:3`)**: This refers to the **columns**. This tells Pandas to grab the first 3 columns (index 0, 1, and 2).

Think of it like GPS coordinates: You are telling Pandas to start at the top-left corner and cut out a specific rectangle of data.

4. Boolean Indexing (Filtering)

Boolean indexing allows you to filter the DataFrame to show only the rows that meet a specific condition. This is essentially the Pandas version of a SQL `WHERE` clause.

Practical Example: Boolean Indexing

Let's filter our `df_sample` to find only the people who are older than 28.

How it works:

1. `df_sample['Age'] > 28` creates a list of True/False values (a 'mask').
2. We pass that mask into the DataFrame `df[...]`.
3. Pandas only returns the rows where the mask is `True`.

In [ ]:

```
# 1. Define the condition
condition = df_sample['Age'] > 28

# 2. Apply the filter
high_age_df = df_sample[condition]

print("--- Original DataFrame ---")
display(df_sample)

print("\n--- Rows where Age > 28 ---")
display(high_age_df)
```


▶ Run in Colab

Multiple Conditions

You can combine conditions using the bitwise operators:

- `&` for **AND**
- `|` for **OR**

Note: You must wrap each condition in parentheses!

In [ ]:

```
# Filter for Salary > 75000 AND City is 'London'
filtered_data = df_sample[(df_sample['Salary'] > 75000) & (df_sample['City'] == 'London')]

display(filtered_data)
```


▶ Run in Colab